Practical No. 3: Implementation of K-Nearest Neighbors (KNN) Classifier

Name: Shantanu Yadav

Roll No: 75

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
df = pd.read_csv('Cancer_Data11.csv')
df
```

Out[2]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mea
0	842302	M	17.99	10.38	122.80	1001.
1	842517	M	20.57	17.77	132.90	1326.
2	84300903	M	19.69	21.25	130.00	1203.
3	84348301	M	11.42	20.38	77.58	386.
4	84358402	M	20.29	14.34	135.10	1297.
...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.
565	926682	M	20.13	28.25	131.20	1261.
566	926954	M	16.60	28.08	108.30	858.
567	927241	M	20.60	29.33	140.10	1265.
568	92751	B	7.76	24.54	47.92	181.

569 rows × 32 columns

In [3]:
```
df.shape
```

Out[3]:
```
(569, 32)
```

In [5]:
```
df.describe()
```

Out[5]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	s
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

8 rows × 31 columns

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64
```

In [7]:
```
df.columns
```

Out[7]:
```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mea
n',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_me
an',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness
_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_
se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
```

In [8]:
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df['diagnosis'] = le.fit_transform(df['diagnosis'])
```

In [9]:
```
df
```

Out[9]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mea
0	842302	1	17.99	10.38	122.80	1001.
1	842517	1	20.57	17.77	132.90	1326.
2	84300903	1	19.69	21.25	130.00	1203.
3	84348301	1	11.42	20.38	77.58	386.
4	84358402	1	20.29	14.34	135.10	1297.
...	...	...	...	...	...	.
564	926424	1	21.56	22.39	142.00	1479.
565	926682	1	20.13	28.25	131.20	1261.
566	926954	1	16.60	28.08	108.30	858.
567	927241	1	20.60	29.33	140.10	1265.
568	92751	0	7.76	24.54	47.92	181.

569 rows × 32 columns

In [11]:
```
X = df.drop('diagnosis',axis = 1)
y = df['diagnosis']
```

In [12]:
```
X
```

Out[12]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothr
0	842302	17.99	10.38	122.80	1001.0	
1	842517	20.57	17.77	132.90	1326.0	
2	84300903	19.69	21.25	130.00	1203.0	
3	84348301	11.42	20.38	77.58	386.1	
4	84358402	20.29	14.34	135.10	1297.0	
...	...	...	...	...	...	
564	926424	21.56	22.39	142.00	1479.0	
565	926682	20.13	28.25	131.20	1261.0	
566	926954	16.60	28.08	108.30	858.1	
567	927241	20.60	29.33	140.10	1265.0	
568	92751	7.76	24.54	47.92	181.0	

569 rows × 31 columns

In [13]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,confusion_matrix,classificatio

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
```

In [19]:
```
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train,y_train)
```

Out[19]:

▾ KNeighborsClassifier

▶ Parameters

In [20]:
```
y_pred_knn = knn_model.predict(X_test)
```

In [21]:
```
accuracy = accuracy_score(y_test, y_pred_knn)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.7340425531914894
```

In [24]:
```
Confusion_matrix = confusion_matrix(y_test,y_pred_knn)

print("Confusion_matrix:",Confusion_matrix)
```

```
Confusion_matrix: [[109  12]
 [ 38  29]]
```

In [25]:
```
print(classification_report(y_test,y_pred_knn))
```

	precision	recall	f1-score	support
0	0.74	0.90	0.81	121
1	0.71	0.43	0.54	67
accuracy			0.73	188
macro avg	0.72	0.67	0.68	188
weighted avg	0.73	0.73	0.71	188